

04: Data Import & Export

ISA 401: Business Intelligence & Data Visualization

2026-09-02

Slides | Class code

The recording is on Canvas, available to ISA 401 instructors. Timestamps before each paragraph are hh:mm:ss into the recording; drag the player there to hear the passage.

Timeline

Start	End	Min	Segment
00:00:01	00:09:20	9	Mentimeter warm-up, three questions on Monday's material
00:09:20	00:20:31	11	Part 1 slides: vectors to data frames, tibbles, lists, the pepper shaker
00:20:31	00:23:59	3	Into RStudio: open the project, new <code>class04.Rmd</code> , YAML header typed
00:23:59	00:30:01	6	Activity 1, examine a job ad, work time
00:30:01	00:36:40	7	Activity 1 debrief, the four expressions worked at the podium
00:36:40	00:46:23	10	Part 2: subsetting a vector, a data frame, the three-bracket table

Start	End	Min	Segment
00:46:23	00:49:32	3	Activity 2, predict the output, work time
00:49:32	00:54:34	5	Activity 2 debrief, six expressions
00:54:34	01:03:34	9	Part 3 CSV: the FRED link, tidyverse install, <code>read_csv()</code> against <code>read.csv()</code>
01:03:34	01:06:26	3	Screen-share debugging, then the editor colour setting
01:06:26	01:08:23	2	Excel setup: <code>readxl</code> , downloading the workbook, its sheets
01:08:23	01:10:06	2	Wednesday's plan, knit, commit and push, close

The twelve segments sum to 70 minutes and the recording ends at 01:10:06, so the table covers it with no unexplained gap. Against the plan: the deck's notes budget about three minutes for the warm-up and set three timed activities across Parts 1 to 3, but the warm-up ran nine minutes and Parts 1 and 2 with their two activities took 45, so Part 3 opened with 15 minutes left and the last two parts went to Wednesday.

Where students struggled

00:05:04 Warm-up question 2, what `crashes_by_day > 2000` returns, was the one the room got wrong. The majority picked “just the days above 2,000” rather than seven TRUE/FALSE values. Fadel worked it from the named vector: the greater-than sign is a logical operator, and applied to a vector it returns TRUE or FALSE for every element, five TRUE for Monday through Friday and two FALSE for Saturday and Sunday, which were both under 2,000. He described it as the equivalent of a for loop that walks every element and asks whether the comparison holds. The option the room chose is what `crashes_by_day[crashes_by_day > 2000]` returns, and saying so pulled the central idea of Part 2 forward by half an hour.

00:02:23 The other two warm-up questions caused no trouble, and the explanations Fadel gave anyway are worth keeping. On question 1, which file to click before opening RStudio, eighteen of twenty-four votes went to `business_intelligence.Rproj`, which is the answer. Opening the project is not strictly required, but double-clicking the `.Rproj` guarantees the working directory starts where the folder is, which matters when the repo is opened on a different computer. Fadel said he had assumed on Monday that everyone's layout was the M drive, then an ISA 401 folder,

then the `business_intelligence` folder, and that it could just as easily be the M drive and then the folder directly. Either way the project opens RStudio inside `business_intelligence`. The `README.md` has no effect on RStudio at all, and `.gitignore` only tells Git which files and folders not to track, so neither starts a session. Question 3, `typeof(c(2020, "Mon"))`, took seventeen of twenty-five votes for `"character"`, which is right.

00:23:45 YAML indentation. Fadel mentioned that a student in one of the two sections had had the indentation of the output options wrong. The fix he demonstrated while typing the header: type the key and its colon first, then press Enter before typing the next key. RStudio indents the new line, and keeps indenting the lines that belong at that level.

00:35:29 After the job-ad debrief he asked for a show of hands, how many were close to at least some of the four answers, then turned to those who were not. He asked twice more, in different words, whether anyone wanted an explanation done differently, and repeated the standing rule that anyone who shows up should leave understanding everything he talked about.

00:50:00 In the predict activity, `crashes_by_day[Sat]` without quotes. The symptom is an object-not-found error rather than Saturday's value; the cause is that unquoted, `Sat` reads as the name of an object in the environment, and no such object exists. Fadel had made the same point earlier while subsetting the vector by name, that the quotes are what separate a name from an object.

00:52:43 `job["salary"]$max` was the expression he expected to catch people, and he flagged that it is different from the one he had typed into the class markdown. The single bracket returns a list, and that outer list contains nothing called `max`. He said he believed it returned NULL rather than an error but was not certain off the top of his head, so he ran it. It returns NULL.

00:54:01 The autocomplete makes the reason visible, and he pointed at it: typing `job["salary"]$` offers only `salary`, because the object at that point is a list of one whose single element is named `salary`. `max` is one level further down, which is why `job["salary"]$salary$max` is what reaches it.

01:03:37 The live debugging moment, and the one worth planning for. A student hit an object-not-found error; Fadel asked them to share their screen, then requested remote control to look. His two usual suspects are that the line creating the object was never run at all, or that Enter was pressed instead of Ctrl+Enter, which inserts a newline in the editor instead of sending the line to R. Here the object had simply never been created. This is the error he had flagged the previous class as one of the most frustrating things in R, because the messages are not plain about what went wrong. The fix was to run the line that creates it, and he thanked the student for sharing, because it produced the exact error he had described.

Questions from the room

00:07:44 A student had asked after the previous class what a vector actually is, and Fadel answered it during warm-up question 3. Think of it as a column of data in an Excel table, with the difference that a spreadsheet column will hold mixed things while R, like most other programming languages, generalises to the most general type that can store everything. In `c(2020, "Mon")` the

number on its own is a double and "Mon" is character, so the only way to preserve both is to store the whole vector as text. Combining integers with doubles generalises to doubles the same way.

01:01:29 A student asked which package had just been installed. Tidyverse, which is a bundle rather than a single package: it brings dplyr, the source of `glimpse()`, and readr, the source of `read_csv()`.

01:02:12 A student asked him to walk back through how he had got the CSV link, and he did it again click by click: the data-source hyperlink in the slide's footer, then Download on the FRED page, then right-click the CSV entry and copy the link address.

01:02:21 In answering he added that the file can be downloaded from the FRED page directly. He thought at first that this download might be aggregated by week rather than daily, then corrected himself that it may well be the same daily data. The reason he did not use the download button on the chart embedded in the slide is that it appeared to offer an XLS rather than a CSV.

Demo notes and gotchas

00:01:35 The warm-up ran on Mentimeter rather than Kahoot. Fadel told the room it had no effect on their grade even though Menti calls it a quiz, and said he had never used Menti for this and was testing it out. The sound coming from the embed was loud enough that he stopped to deal with it before talking over the first set of results.

00:08:43 The leaderboard is the last Menti slide and he nearly skipped it, remarking that he probably should have shown it. He congratulated the top scorer, said there were no points this time, and said there would be points next time.

00:09:43 The session was framed around two errors he expects to cost students time, one in Part 1 and one in Part 3. His stated goal for the day was that the room leave knowing both and how to address them.

00:10:06 The Lego picture on the data frame slide carried the four vector types: logical as the grey and yellow tower, the output of something like `crashes_by_day > 2000`; categorical as a fixed set of labels, day of the week being Monday through Sunday and nothing else; numeric as a double or an integer; and character as the open one, where the number of possible values is effectively unlimited. He used the same picture again for the one-dimensional point, that pulling from a vector needs only one thing inside the brackets, a single number or a vector of numbers.

00:12:36 For the rule that all columns of a data frame must be the same length he gave a practical example that is not on the slide: web scraping, where two things come back at different lengths and the attempt to build a data frame fails, which is the signal that a cleaning step is needed first. He noted that Python errors the same way and for the same reason, because neither language knows how to line the rows up.

00:14:00 A matrix aside, not in the deck. Fadel raised matrices only because an AI assistant may well suggest one. A matrix looks like a data frame but can hold only one type, so combining numbers with characters turns the entire matrix to character. He drew a second contrast about named columns and hedged it in the same breath.

00:14:21 Two tibble-versus-data-frame differences he added for anyone who had not used a tibble: a tibble prints its size, its column names and its column types before any of the data, and extracting a one-by-one element gives a single element from a data frame but a one-by-one tibble from a tibble.

00:14:49 The four questions he put to any loaded data frame are how many rows, how many columns, what the columns are called, and what type each one is. `nrow()` and `ncol()` answer the first two, `colnames()` or `names()` the third. `typeof()` on a data frame is the odd one, reporting "list" rather than anything about a particular column, so asking about one column means writing something like `crashes_df$day`.

00:16:07 He then said he tends to ignore all of those and load dplyr for `glimpse()` instead, which gives the row and column counts, the names, the types and the first few values in one call. For anyone who has used R before it is similar to `str()` and just prints more readably; he left the choice open. He explained his colour convention in the same breath, since it shows up both on the slides and in his editor: mustard yellow is a package or library name, the bluish colour is a function name. He said he thought everyone in this section had used R before, which was not the case in his other section.

00:16:50 Lists were introduced as what usually comes back from an API, precisely because nothing in them has to match: a list can hold another list, a data frame, and vectors of different lengths at once. The slide's `example_list` holds four elements, the integers 1, 2, 3, a single character "a", a logical vector of length 3, and a numeric vector of length 2.

00:18:20 The pepper shaker is the mental model he leaned on hardest, and the place he put the first of the day's two frustrating errors. The shaker is the list and the packets are the elements. `x[1]` gets the first packet but leaves it in the container, so what comes back is still a list; `x[[1]]` hands over the packet; `x[[1]][[1]]` is the pepper. He tied it to why it matters: a function such as `mean()` takes a vector and not a list, so the container has to be opened before the function will accept what is inside. He came back to the shaker twice, at the job-ad debrief and again at the three-bracket summary table.

00:20:59 RStudio opened on whatever project was last used, which he pointed out was not the recommendation he had just made in warm-up question 1. He double-clicked the project file for this section and pointed at the top right corner of RStudio, where the project name appears. He teaches two sections, so he has two projects to choose between; a student with one class in this lab has only one.

00:21:18 He asked whether the room could read the slides from where they were or wanted him to switch to RStudio. Someone in the room answered, and he went to RStudio full screen for the header typing. When he wanted the slide and RStudio visible at once, a few minutes later, getting the two windows next to each other took several attempts, and he said out loud that the computer was not cooperating. Budget a moment for it.

00:21:50 The markdown was built the way Monday's was: File, New File, R Markdown, then delete everything after the setup chunk and type the output options from the previous class. It was saved

with File, Save As into the `markdowns` folder as `class04.Rmd`; he noted that the deck he had made that morning calls it Class 4, so he kept that name.

00:25:05 The instructions he gave while the first activity clock ran: insert a chunk with the insert button, give it a name such as `activity01`, copy the list definition off the slide rather than retyping it, and run with Ctrl+Enter, which works with the cursor anywhere on the line, at the end as well as the beginning.

00:30:01 He said he had locked the answers panel so the room could not read ahead, and promised to share the answer code at the end of class.

00:34:05 A habit he narrated as he worked: test the expression in the Console, confirm it returns what was expected, and only then write it into the chunk.

00:35:12 He rearranged the panes so the whole chunk was visible at once, and mentioned bracket colouring as a reading aid, since with two pairs on screen you can see which opening bracket goes with which closing one. He was explicit that it has no effect on the code.

00:36:40 He clicked Knit before going back to the deck, so the HTML would generate in the background while he talked.

00:38:34 Rather than assert that the dollar sign does not work on a vector, he opened a chunk and ran the experiment. It does not work. He said plainly that this is what the Console is for when you are not sure.

00:40:38 Two reading rules he wanted carried away: a square bracket means extracting from a data structure, whatever that structure is, and parentheses mean applying a function and giving it arguments. For anyone who did Python in ISA 104, he added that Python starts counting from 0 and R starts from 1.

00:45:22 The three-bracket summary slide ends on two prohibitions worth repeating: no dollar sign on a vector at all, and the dollar sign on a list only when the elements have names, which sometimes they do not. He skipped double brackets on vectors as rarely needed.

00:46:51 The second activity was framed as prediction rather than execution. Copying the code and checking was allowed, but the short clock was there to force reasoning from what had just been covered. He worked all six aloud in the debrief, corrected himself on the first one, since `crashes_by_day[c(2, 4)]` is Tuesday and Thursday rather than Wednesday and Friday, and had to run the sixth to settle it.

00:56:26 The click order that produced the CSV address: the data-source hyperlink in the slide's footer opens the FRED page for the series, Download on that page opens a menu offering CSV, Excel, an image and PowerPoint, and right-clicking the CSV entry copies the link address. That address, in quotes, is what `read_csv()` takes.

00:57:35 The tidyverse install went into the Console on purpose, so the install line would not sit in the markdown. Note the quotes around `"tidyverse"`: it is not an object in the environment, so it has to be a string. RStudio offered to restart R and he declined, since nothing had been loaded in that session yet.

00:58:31 `read_csv()` was a function the class had not used, so he put the cursor on it and pressed F1 to open the help page, then widened the pane to read it. The arguments he pointed at: `file`, `col_names` defaulting to TRUE, white space trimmed, `skip` at 0, and type guessing from a minimum of a thousand rows. The line that matters for the demo is the description of `file`, which says it can be a URL.

00:59:28 The pasted address is very long and does not fit the editor width, and he said so. The deck's own version of this demo does it differently: it builds a short URL with `paste0()` from the series id and a date range, and follows the read with a one-line `plot()`. Neither was used.

01:00:08 Running the readr line prints rather than staying silent: it reports how many rows and columns it read and the types it assigned, and reports that the date column came back as a date.

01:00:37 The comparison is the reason the file is read twice. Both objects came back with the same 1,827 rows and the same two variables, but `glimpse()` shows the readr version holding the date column as a date and the value as a double, while the base R version left the date column as character. The dashes in the date are why base R keeps it as text.

01:05:49 The colouring that makes package and function names appear differently is optional and cosmetic. It lives under Tools, Global Options; Fadel was briefly unsure whether it sat under Appearance or Code, settled on Code, then Display, where he had two boxes ticked. He left it to the room whether to apply it.

01:06:34 Excel needs its own install. He said the package name aloud as “read Excel” and then spelled out that the package itself is `readxl`. He put up the slide on reading an Excel file, then stepped back one slide to the download-first slide, and moved on to Canvas without working through either.

01:07:15 The workbook comes from Canvas, from the module item for this class, as `occupation.xlsx`. Download it rather than pointing code at a URL.

01:07:40 His strong recommendation before any code is written is to open the downloaded workbook in Excel, because Excel data tends to be messy and the messiness is structural. A workbook can hold multiple sheets, which is why a CSV is called a flat file and a workbook is not. This one opens on an Index sheet listing the tables, and the tabs along the bottom are that Index plus eleven numbered tables, twelve sheets in all; the numbering runs 1.1 to 1.12 with no Table 1.7. Table 1.3 is the one the class wants, and opening it shows the second problem: row 1 is a title written for a human reader, and the column names sit in row 2. He deliberately did not push further, since reading the workbook properly is Wednesday's work.

2025 National Employment Matrix title	2025 National Employment Matrix code	Employment, 2025	Employment, 2035	Employment change, numeric, 2025-35	Employment change, percent, 2025-35	Median annual wage, dollars, 2025
Total, all occupations	00-0000	170,280.8	178,198.5	5,917.6	3.5	50,980
Nurse practitioners	29-1171	336.3	474.1	137.8	41.0	132,300
Solar photovoltaic installers	47-2231	31.1	42.4	11.3	36.5	53,140
Data scientists	15-2051	275.6	371.0	95.4	34.6	120,230
Wind turbine service technicians	49-9081	11.8	15.3	3.5	29.5	64,120
Medical and health services managers	11-9111	640.4	795.5	155.1	24.2	123,860
Physical therapist assistants	31-2021	114.1	140.4	26.2	23.0	68,380
Psychiatric technicians	29-2053	161.5	197.5	36.0	22.3	45,130
Computer and information research scientists	15-1221	38.6	47.0	8.4	21.8	140,300
Occupational therapy assistants	31-2011	52.2	63.4	11.2	21.5	72,300
Ophthalmic medical technicians	29-2057	72.5	88.0	15.5	21.4	45,570
Physician assistants	29-1071	168.9	204.6	35.7	21.1	135,880
Information security analysts	15-1212	192.9	233.4	40.6	21.0	129,180
Occupational health and safety technicians	19-5012	31.2	37.2	6.1	19.4	61,560
Hearing aid specialists	29-2092	11.2	13.4	2.2	19.4	65,160
Endorsement specialists	16-1041	12.8	15.2	2.4	18.7	87,220

Figure 1: Table 1.3 open in Excel: row 1 is the table's title, the column names sit in row 2, and the sheet tabs along the bottom start at Index and run through the numbered tables

01:09:08 The end-of-class ritual, now with data in it: knit the document, then commit and push from GitHub Desktop. Fadel used `end of class04` as the commit summary and explained that committing stores that version and pushing puts it on GitHub, so the work is available in later classes.

Code as taught

The class RMD was built live in front of the room, nothing prepared in advance. The two things not typed out are the `job` list definition, which Fadel told the room to copy off the slide rather than retype, and the FRED URL, which was pasted from the browser. Several expressions were tried in the Console and never made it into the file.

The YAML header, as typed:

```
title: 'Class 04: R Data Structures'
author: "Fadel M Megahed"
date: "`r Sys.Date()`"
output:
  html_document:
    toc: true
    toc_float: true
    number_sections: true
    code_folding: show
    code_download: true
```

00:23:59 The first chunk came out of the job-ad activity, two or three minutes on the clock and four expressions against the `job` list: the company name as a value rather than as a list, the maximum salary as a number, the maximum salary again using only square brackets, and the names of the elements.

00:30:01 The debrief was worked at the podium. Position can be confirmed by expanding `job` in the Environment pane, which is also where the element names can be read without calling `names()`. The autocomplete carried one of the points: after `job$salary$` RStudio offers the named things inside, which is only possible because the sublist has names, and that is why the dollar-sign path works at all.

The chunk, as it ended up:

```
#| eval: false
job = list(title = "Research Data Scientist, Magic Eye",
  company = "Google", salary = list(min = 147000, max = 210000) )

# 1. The company name.
company = job[[2]] # if you were to use one sq bracket, job[2]
company_alt = job$company

# 2. The maximum salary.
max_salary = job$salary$max

# 3. The maximum salary again, using only [[ ]].
max_salary_alt = job[[3]][[2]]

job['salary']$salary$max

# 4. The names of the elements in job.
names(job)
```

Line 6

Company is the second element, and the double bracket opens the container, so this is the string "Google". A single bracket here would have given a list of one instead.

Line 7

The same thing by name.

Line 10

Two steps down: in `job`, take `salary`, then take `max`.

Line 13

The same path written with positions: the third element of `job`, then the second element of that.

Line 15

Starting with a single bracket instead. `job['salary']` is still a list, so `salary` has to be named again to get inside it before `max` is reachable.

Line 18

Title, company, salary.

00:35:51 The name he gave to what the activity had just done is subsetting: getting a subset of the data, and making sure that subset comes out in the format the next step needs. Part 2 then went back over it properly, since Part 1 had used subsetting without explaining it, and only for a list.

00:37:11 The second chunk is the vector subsetting section. The vector is real data, unique crashes in Cincinnati in 2025 summed by day of the week across the whole of last year, and it is named, so `crashes_by_day[2]` returns the second day, Tuesday, with its name attached, and `crashes_by_day["Fri"]` does the same for Friday. To pull more than one element the brackets need a vector, and when the wanted elements sit next to each other the colon does it.

```
#| eval: false
crashes_by_day = c(2020, 2241, 2327, 2285, 2380, 1866, 1664)
names(crashes_by_day) = c("Mon", "Tue", "Wed", "Thu",
                          "Fri", "Sat", "Sun")

crashes_by_day[c("Wed", "Fri")] # pulling elements by their name
crashes_by_day[c(3, 5)]

crashes_by_day[1:5] # : only works for elements in a row [by their index]
crashes_by_day[] # returns all the elements --> no diff than crashes_by_day
```

Line 6

Two elements by name. The quotes are required: without them R reads `Wed` as the name of an object in the environment, and there is no such object.

Line 7

The same two elements by position. Wednesday is the third day, Friday the fifth.

Line 9

The colon only works when the elements are in a row, and it works on positions, not names.

Line 10

Empty brackets return everything, which is no different from typing the object's name.

00:43:25 No data frame chunk was typed; that material stayed on the slides. What Fadel added there is that `crashes_df[["crashes"]]` and `crashes_df$crashes` do the same thing, both pulling one column out as a vector, and that the two-dimensional form takes rows, a comma, then columns, where either side can be a single position, a range such as `1:2`, a vector of positions such as `c(1, 3)`, column names such as `c("day", "crashes")`, or a logical condition.

00:54:34 The third chunk is the CSV read. Three formats were promised for the day, CSV, Excel and JSON. `read_csv()` reads data on the machine and data on the web, as long as what it is pointed at is actually a CSV and not, say, a folder containing one; it works the same way as base R's

`read.csv()`, and Fadel said he prefers the underscore version because it guesses column types. Excel files have to be on the machine. JSON, the semi-structured format from the earlier class, can come from either. Only the CSV was reached.

00:55:47 The file is the Indeed job postings index for Ohio, served by FRED, with roughly five years of data. It is an index rather than a count: February 1, 2020 is set to 100, so a value above 100 means more job postings than there were on that date and a value below 100 means fewer. The chunk reads it twice, once with `readr` and once with base R, so the two can be compared.

```
#| eval: false
oh_jobs = readr::read_csv(
  file="https://fred.stlouisfed.org/graph/fredgraph.csv?bgcolor=%23ebf3fb&
chart_type=line&drp=0&fo=open%20sans&graph_bgcolor=%23ffffff&height=450&mode=
fred&recession_bars=on&txtcolor=%23444444&ts=12&tts=12&width=1140&nt=0&thu=
0&trc=0&show_legend=yes&show_axis_titles=yes&show_tooltip=yes&id=IHLIDXUSOH&
scale=left&cosd=2021-08-28&coed=2026-08-28&line_color=%230073e6&link_values=
false&line_style=solid&mark_type=none&mw=3&lw=3&ost=-99999&oet=99999&mma=0&
fml=a&fq=Daily%2C%207-Day&fam=avg&fgst=lin&fgsnd=2020-02-01&line_index=1&
transformation=lin&vintage_date=2026-09-02&revision_date=2026-09-02&nd=2020-
02-01"
)

oh_jobs_baser= read.csv("https://fred.stlouisfed.org/graph/fredgraph.csv?
bgcolor=%23ebf3fb&chart_type=line&drp=0&fo=open%20sans&graph_bgcolor=%23
ffffff&height=450&mode=fred&recession_bars=on&txtcolor=%23444444&ts=12&tts=12&
width=1140&nt=0&thu=0&trc=0&show_legend=yes&show_axis_titles=yes&show_tooltip=
yes&id=IHLIDXUSOH&scale=left&cosd=2021-08-28&coed=2026-08-28&line_color=%
230073e6&link_values=false&line_style=solid&mark_type=none&mw=3&lw=3&ost=-
99999&oet=99999&mma=0&fml=a&fq=Daily%2C%207-Day&fam=avg&fgst=lin&fgsnd=2020-
02-01&line_index=1&transformation=lin&vintage_date=2026-09-02&revision_date=
2026-09-02&nd=2020-02-01")

dplyr::glimpse(oh_jobs)
dplyr::glimpse(oh_jobs_baser)
```

Line 3

The copied link address, in quotes. It is long enough that it runs well past the editor width, which Fadel remarked on as he pasted it. Mustard yellow is the package, blue is the function, in Fadel's editor colours.

Line 6

The same URL read with base R's `read.csv()`, so the two can be compared.

Line 8

`glimpse()` on both is the habit, and here it is also the comparison.

For next time

Fadel's own remarks about the session come first, then the note's suggestions, then what is still unconfirmed.

00:08:43 On the warm-up: he said he probably should have shown the last Menti slide, the leaderboard, and that there were no points this time but there would be points next time.

00:20:59 On opening RStudio: he noted that he had not followed the recommendation he had just made in warm-up question 1, since RStudio came up on the project he had used last rather than from the project file.

00:54:34 On pacing: he said out loud that the class was doing well on time, then immediately took it back.

00:56:26 On the slides: he said the FRED chart embedded in the deck had got squished, that he did not know why, and that he could probably fix it later.

01:08:23 On Wednesday: he said he would not rush the Excel workbook and would talk about it then, that he did not plan to make any new slides, and that he would finish importing data and then use that as an excuse to look at getting insights from imported data with AI, using a package that lets you ask an OpenAI model questions about the data repeatedly from inside R.

01:09:55 On the session as a whole: he thanked the people who asked questions, said it had been really helpful, and repeated the standing invitation to say so if he sped up through something.

Suggestion: budget nine minutes for the warm-up rather than three. Two of the three questions needed a full explanation, and the leaderboard came after them.

Suggestion: the deck and the demo disagree about how the FRED URL is built. The deck assembles a short one with `paste0()` from the series id and a date range, and follows the read with a one-line `plot()`; the class pasted the full copied link address and skipped the plot. Settle which one the class RMD should carry before the demo is run again.

Suggestion: the job-ad activity was given two or three minutes and ran six, and its debrief ran seven. If the deck's three-minute countdown is kept, plan the period around the debrief rather than the clock.

Suggestion: what is left is more than one Wednesday. The Excel read with `sheet`, `skip` and `n_max`, the `../data/` path convention and its animated slide, JSON, the Import Heist and Part 4 on export are all still to come. Decide in advance which of them moves further out.

Open: what the matrix and data frame contrast at 00:14:00 was meant to say. The transcript garbles it and it is on no slide.

Open: the name of the package for Wednesday's AI demo.

Open: the two checkboxes under Tools, Global Options, Code, Display that give function and library highlighting.

Open: whether Part 4 on export and the Import Heist move to Wednesday or later, so the Class 05 notes can pick them up.

Open: how much of the room was close to the job-ad answers when the show of hands was taken.

Materials

- Slides
- Class code